Trabajo Práctico Integrador Grupal

Computación Aplicada – Universidad de Palermo

Integrantes: Danuvio Heredia · Mia Fernandez · Valentina Alvarez

Servidor: Debian GNU/Linux 12 (bookworm) · Hostname: TPServer · IP: 192.168.0.200/24

Índice

1. Configuración del entorno
2. Servicios: actualización del sistema, SSH, Apache con PHP y MariaDB
3. Configuración de red
4. Almacenamiento
5. Backup
6. Entregables
7. Verificación integral del trabajo
8. Preguntas de repaso
9. Bibliografía

1. Configuración del entorno

1.1 Ensamblado e importación de la máquina virtual

La máquina virtual se distribuye dividida en nueve volúmenes de un archivo RAR multiparte (`TPVMCA.part01.rar` a `TPVMCA.part09.rar`). En un archivo multiparte, el programa de descompresión abre el primer volumen y busca los siguientes por su nombre exacto; por eso, si el navegador agrega un sufijo al descargarlos —por ejemplo `TPVMCA.part01(1).rar`—, la secuencia se rompe y la extracción falla. Restituido el nombre original de cada parte, al abrir `part01` se obtiene el archivo `TPVMCA.ova`.

Un archivo `.ova` (*Open Virtual Appliance*) empaqueta la definición de la máquina (procesadores, memoria, controladoras, placa de red) junto con sus discos virtuales. Se incorpora en VirtualBox desde **Archivo → Importar servicio virtualizado**. La máquina importada tiene 2 CPU, 2048 MB de RAM y dos discos: uno de 10 GB que contiene el sistema (`/` y la partición de intercambio) y otro de 8 GB montado en `/home`.

La placa de red de la máquina está configurada en modo **adaptador puente**: la VM se conecta a la misma red física que la computadora anfitriona y obtiene una dirección de ese rango, como si fuera un equipo más de la red. Ese modo es el que permite cumplir la nota de la consigna, que exige probar los servicios desde la máquina física. Al importarla, el puente apuntaba a una interfaz del equipo donde se

creó la VM (`wlp3s0`), que no existe en otra computadora; por eso hay que seleccionar la placa de red real del anfitrión en **Configuración → Red**.

> Fuente: consigna del TP (punto 1.1); apunte *Descarga de Linux para la instalación* (uso de VirtualBox).

1.2 Blanqueo de la contraseña de root

La consigna parte de una contraseña de root desconocida. El procedimiento aprovecha que, durante el arranque, el gestor de arranque GRUB permite editar los parámetros que recibe el kernel. Si se le indica `init=/bin/bash`, el kernel no inicia el sistema de inicialización habitual (systemd), que es el que luego presenta la pantalla de login, sino que ejecuta directamente un intérprete de comandos con privilegios de root. El parámetro `rw` monta la partición raíz con permiso de escritura, necesario para que el cambio de contraseña quede guardado en `/etc/shadow`.

1. Reiniciar la VM y, mientras arranca, presionar una flecha del teclado para detener el menú de GRUB.
2. Con la entrada *Debian GNU/Linux* seleccionada, presionar e para editarla.
3. Ubicar la línea que comienza con `linux /boot/vmlinuz-…`, ir al final con `Fin` y agregar `rw init=/bin/bash`.
4. Arrancar con `Ctrl`+X. Aparece un prompt `root@(none):/#`.
5. Cambiar la contraseña y continuar el arranque normal:

```
passwd                  # pide la nueva contraseña dos veces: palermo
exec /sbin/init         # reemplaza la shell por el sistema de inicio y el arranque continúa
```

Una forma equivalente y no interactiva es `echo 'root:palermo' | chpasswd`, que recibe por entrada estándar pares *usuario:contraseña*. La edición en GRUB es temporal: afecta solo a ese arranque y no modifica la configuración permanente.

> Este procedimiento muestra por qué el acceso físico a un equipo compromete su seguridad: quien puede editar el arranque obtiene control total. En servidores reales se mitiga protegiendo GRUB con contraseña y cifrando el disco.

> Fuente: consigna del TP (punto 1.2); apunte *El standard FHS* (/boot y GRUB). El método de edición de GRUB no figura en los apuntes disponibles; corresponde a la documentación de GRUB y Debian.

1.3 Nombre de host: TPServer

El *hostname* es el nombre interno con el que el equipo se identifica en la red. Se define con `hostnamectl`, que actualiza `/etc/hostname`; además conviene actualizar `/etc/hosts`, donde Debian asocia el nombre del equipo a la dirección 127.0.1.1 para que se resuelva localmente.

```
hostnamectl set-hostname TPServer
nano /etc/hosts         # la línea queda:  127.0.1.1   TPServer
hostname                # verificación: TPServer
```

> Fuente: consigna del TP (punto 1.3); apunte *Descarga de Linux* (definición de hostname); apunte *El standard FHS* (/etc).

2. Servicios

2.1 Actualización de Debian 11 (bullseye) a Debian 12 (bookworm)

Debian es una distribución de tipo *standard release*: agrupa los cambios en versiones estables numeradas, y permite pasar de una versión mayor a la siguiente sin reinstalar, mediante un **upgrade de distribución**. El gestor de paquetes APT obtiene el software de los repositorios declarados en `/etc/apt/sources.list`; por lo tanto, cambiar de versión consiste en apuntar esos repositorios a la nueva distribución y reemplazar todos los paquetes instalados.

Cada línea de `sources.list` sigue el formato `deb uri distro componentes`, donde *distro* es el nombre de la versión (bullseye, bookworm) y los componentes indican el tipo de software: `main` (libre, cumple las DFSG), `contrib` (libre con dependencias no libres) y `non-free-firmware` (firmware no libre, separado a partir de Debian 12).

El archivo quedó con los repositorios de bookworm:

```
deb http://deb.debian.org/debian bookworm main contrib non-free-firmware
deb http://security.debian.org/debian-security bookworm-security main contrib non-
free-firmware
deb http://deb.debian.org/debian bookworm-updates main contrib non-free-firmware
```

Secuencia de actualización:

```
cat /etc/debian_version        # versión inicial: 11.x
apt update                     # relee los índices de los repositorios
apt upgrade                    # actualiza lo instalado dentro de la versión actual
nano /etc/apt/sources.list     # reemplazar bullseye por bookworm
apt update                     # índices de bookworm
apt full-upgrade               # actualización completa: puede instalar y quitar
paquetes
apt autoremove                 # elimina dependencias que quedaron sin uso
reboot
cat /etc/debian_version        # 12.15
uname -r                       # 6.1.0-54-amd64 (kernel de Debian 12)
```

La diferencia entre `upgrade` y `full-upgrade` es central: `upgrade` nunca elimina paquetes ni instala paquetes nuevos, por lo que no alcanza para un cambio de versión, en el que cambian bibliotecas y dependencias; `full-upgrade` sí resuelve esos cambios.

Inconvenientes encontrados y su resolución

- **Repositorios desactualizados.** El `sources.list` original apuntaba a un espejo que ya no respondía y a una ruta de seguridad con formato antiguo (`bullseye/updates`). Se reemplazó

por los repositorios oficiales indicados arriba. Para la línea de seguridad se usa la ruta oficial `security.debian.org/debian-security`.

- **Configuración de GRUB durante la actualización.** El paquete `grub-pc` pregunta en qué disco instalar el gestor de arranque. Hay que indicar el disco del sistema (`/dev/sda`); si la pregunta queda sin responder, la configuración del paquete falla y se completa luego con `dpkg --configure -a`.
- **Preguntas sobre archivos de configuración modificados** (por ejemplo `sshd_config` o `sudoers`): se conserva la versión local instalada.
- **Resolución por IPv6.** En la red utilizada, los nombres de los repositorios resolvían primero a direcciones IPv6 sin conectividad. Se forzó APT a usar IPv4 con el archivo `/etc/apt/apt.conf.d/99force-ipv4`, que contiene `Acquire::ForceIPv4 "true";`. Es un ajuste propio de esa red, no un paso general del procedimiento.

> Fuente: apunte *Gestor de paquetes* (sources.list, APT y dpkg, upgrade de sistema operativo); consigna del TP (punto 2.1).

2.2 Servicio SSH con autenticación por clave pública

SSH (*Secure Shell*) es un protocolo y un programa para administrar equipos remotos a través de un canal cifrado. Funciona con un modelo cliente-servidor: el cliente inicia la conexión y el servidor (`sshd`) escucha, por defecto, en el puerto TCP 22. El cliente suele venir instalado; el servidor se instala con el paquete `openssh-server`.

```
apt-get install openssh-server
systemctl enable ssh           # inicio automático en cada arranque
systemctl status ssh
```

Autenticación por clave pública/privada

En este método el usuario posee un par de claves. La **clave pública** se copia al servidor, en `~/.ssh/authorized_keys` del usuario con el que se ingresa; la **clave privada** queda en poder del cliente y no se comparte. Al conectarse, el servidor plantea un desafío que solo puede resolverse con la clave privada correspondiente, de modo que la autenticación se completa sin transmitir contraseñas.

La consigna indica que el par de claves está disponible en el home de root. El material adicional del TP (`/root/Material_Adicional_TPVMCA.tar.gz`) contiene `clave_publica.pub` y `clave_privada.txt`, junto con `index.php`, `logo.png` y `db.sql`:

```
cd /root
tar -xzf Material_Adicional_TPVMCA.tar.gz
cp Material_Adicional_TPVMCA/* /root/
mkdir -p /root/.ssh && chmod 700 /root/.ssh
cat /root/clave_publica.pub >> /root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
```

Los permisos restrictivos son obligatorios: si `~/.ssh` o `authorized_keys` pueden ser escritos por otros usuarios, `sshd` ignora las claves por seguridad.

En `/etc/ssh/sshd_config` se configuraron las directivas indicadas en el apunte:

```
PermitRootLogin yes
PubkeyAuthentication yes
```

```
systemctl restart ssh
```

`PermitRootLogin` trae por defecto el valor `prohibit-password`, que admite a root solo mediante clave. El apunte lo cambia a `yes` con fines prácticos, lo que habilita a root tanto con clave como con contraseña. `AuthorizedKeysFile` conserva su valor por defecto, `.ssh/authorized_keys`.

Prueba desde la máquina física (en Windows, desde PowerShell), usando la clave privada del TP:

```
ssh -i clave_privada.txt root@192.168.0.200
```

Si la conexión se establece sin pedir contraseña, la autenticación por clave funciona. Cuando el par de claves se genera desde cero, el procedimiento del apunte es `ssh-keygen` (crea `id_rsa` e `id_rsa.pub` en `~/.ssh`) seguido de `ssh-copy-id usuario@servidor`, que agrega la clave pública al `authorized_keys` remoto.

> Fuente: apunte *SSH – Secure Shell*; consigna del TP (punto 2.2).

2.3 Servidor web Apache con soporte para PHP

Apache HTTP Server atiende las peticiones HTTP (puerto 80) y entrega el contenido del directorio configurado como raíz del sitio. Por sí solo sirve archivos estáticos; para ejecutar código PHP necesita un módulo que interprete los archivos `.php` antes de enviar la respuesta. En Debian 12 se instalan:

```
apt-get install apache2 php libapache2-mod-php php-mysql
php -v                          # PHP 8.2.x: cumple el requisito de versión 7.3 o
superior
systemctl enable --now apache2
```

- `apache2`: el servidor web.
- `php` y `libapache2-mod-php`: el intérprete y el módulo que lo integra a Apache.
- `php-mysql`: la extensión *mysqli*, que `index.php` usa para conectarse a la base de datos.

La configuración para que el servidor entregue `index.php` y `logo.png` desde su ubicación definitiva se desarrolla en el punto 4.3, porque la consigna de almacenamiento exige alojarlos en `/www_dir`.

> Fuente: consigna del TP (punto 2.3). No se dispone de apunte de la materia sobre Apache; los paquetes y comandos corresponden a la documentación oficial de Debian 12.

2.4 Base de datos MariaDB y carga de db.sql

MariaDB es un motor de base de datos relacional compatible con MySQL. Se instala y se habilita como servicio:

```
apt-get install mariadb-server
systemctl enable --now mariadb
```

El script `db.sql` crea la base `ingenieria`, el usuario `lcars` con contraseña `NCC1701D` y privilegios sobre esa base, y tres tablas con sus datos: `alumnos` (legajo, apellido, nombres), `modulos` (código y nombre de materia) y `notas` (legajo, código de módulo, nota). Se carga redirigiendo el archivo a la entrada estándar del cliente:

```
mysql -u root < /root/db.sql
mysql -u root -e "SHOW DATABASES;"
mysql -u root ingenieria -e "SELECT * FROM alumnos;"
```

En Debian, el usuario root de MariaDB se autentica por *socket* Unix cuando se invoca como root del sistema, por eso no pide contraseña. El símbolo `<` es la redirección de entrada vista en el apunte de redireccionamiento: el comando lee desde el archivo en lugar del teclado.

`index.php` se conecta a `localhost` con el usuario `lcars` y ejecuta una consulta que relaciona las tres tablas (legajo → alumno, código de módulo → materia) para mostrar legajo, apellido, nombre, materia y nota en una tabla HTML encabezada por `logo.png`. Con la base cargada, la página lista las 15 notas del script.

> Fuente: consigna del TP (punto 2.4); apunte *Redireccionamiento y Pipes* (operador <). No se dispone de apunte específico sobre MariaDB.

3. Configuración de red

Una dirección IPv4 tiene 32 bits, escritos como cuatro octetos decimales. La **máscara de red** indica qué parte de la dirección identifica la red y qué parte al equipo: 255.255.255.0 (notación CIDR `/24`) reserva los tres primeros octetos para la red. El **gateway** es el equipo, normalmente el router, al que se envían los paquetes destinados a otras redes, incluido Internet.

La consigna pide una IP estática dentro del rango de la máquina física. El anfitrión estaba en la red 192.168.0.0/24, con gateway 192.168.0.1; se eligió **192.168.0.200**, tras comprobar que ningún otro equipo la usaba. En Debian la configuración se declara en `/etc/network/interfaces`:

```
# The loopback network interface
auto lo
iface lo inet loopback

# The primary network interface
auto enp0s3
allow-hotplug enp0s3
iface enp0s3 inet static
        address 192.168.0.200
        netmask 255.255.255.0
        gateway 192.168.0.1
```

- `auto`: levanta la interfaz al iniciar el sistema.
- `allow-hotplug`: la levanta cuando el sistema detecta el dispositivo.
- `iface enp0s3 inet static`: interfaz `enp0s3`, protocolo IPv4, configuración manual (en lugar de `dhcp`).
- `address`, `netmask`, `gateway`: los tres campos exigidos por la consigna.

```
ip a                            # identificar el nombre de la interfaz (enp0s3)
systemctl restart networking    # aplicar los cambios
ip -br a                        # verificar: enp0s3 UP 192.168.0.200/24
ping -c 3 192.168.0.1           # conectividad con el gateway
```

Con asignación estática, el servidor DNS deja de obtenerse por DHCP; por eso se declara en `/etc/resolv.conf` (`nameserver 192.168.0.1`). Sin un DNS, el equipo no traduce nombres como `deb.debian.org` a direcciones IP y APT no puede descargar paquetes.

> La IP estática pertenece a la red donde se configuró. Si la VM se usa en otra red (por ejemplo, otro router con rango 192.168.1.x), hay que adaptar `address` y `gateway` al nuevo rango; de lo contrario la VM quedará incomunicada.

> Fuente: apunte *Redes* (IPv4, máscara, gateway, DNS, DHCP y archivo interfaces); consigna del TP (puntos 3.1 y 3.2).

4. Almacenamiento

4.1 Incorporación de un disco de 10 GB

Con la máquina apagada: **Configuración → Almacenamiento → Controlador SATA → Agregar disco duro → Crear**, formato VDI, tamaño 10 GB y sin la opción de reservar todo el espacio de entrada (*pre-allocate full size*). Al encender la VM, el disco aparece como un nuevo dispositivo de bloques:

```
lsblk
sda      10G  disk           (sistema: sda1 /  y  sda5 swap)
sdb       8G  disk           (sdb1 /home)
sdc      10G  disk           ← disco nuevo, sin particiones
```

En Linux cada dispositivo de hardware se representa como un archivo dentro de `/dev`; los discos son archivos de bloques (`sda`, `sdb`, `sdc`) y sus particiones se numeran (`sdc1`, `sdc2`).

> Fuente: apunte *Introducción a filesystem* (incorporación de disco en VirtualBox); apunte *El standard FHS* (/dev y archivos de bloques).

4.2 Particiones estándar tipo 83

El tipo 83 es el identificador de partición *Linux* en las tablas de particiones MBR (DOS). Se crearon dos particiones primarias en `/dev/sdc`: la primera de 3 GB para `/www_dir` y la segunda de 6 GB para `/backup_dir`. Con `fdisk`, de forma interactiva:

```
fdisk /dev/sdc
  n → p → 1 → Enter → +3G      (partición 1, primaria, 3 GB)
  n → p → 2 → Enter → +6G      (partición 2, primaria, 6 GB)
  p                            (verificar: ambas con Id 83, tipo "Linux")
  w                            (escribir la tabla y salir)
```

La misma tabla se puede crear de forma no interactiva con `sfdisk`, que recibe la descripción por entrada estándar; es el método que se utilizó en la VM:

```
printf 'label: dos\n,3G,83\n,6G,83\n' | sfdisk /dev/sdc
```

Una partición solo reserva espacio; para guardar archivos necesita un sistema de archivos. Se usó ext4, que es journalizado: registra cada operación en el *journal* antes de aplicarla, lo que permite recuperar un estado consistente tras un corte de energía.

```
mkfs.ext4 /dev/sdc1
mkfs.ext4 /dev/sdc2
mkdir /www_dir /backup_dir
fdisk -l /dev/sdc
/dev/sdc1   2048   6293503   6291456   3G  83 Linux
/dev/sdc2   6293504 18876415 12582912  6G  83 Linux
```

La nota de la consigna pide conservar una copia de `/proc/partitions`. `/proc` no contiene archivos reales sino información que el kernel genera en memoria, por lo que su contenido se pierde al apagar. Con la redirección `>` se guarda la salida en un archivo permanente:

```
cat /proc/partitions > /opt/particion
```

> Fuente: apunte *Introducción a filesystem* (ext4 y journaling); apunte *El standard FHS* (/proc, /opt); apunte *Redireccionamiento y Pipes* (operador >); consigna del TP (punto 4.2 y nota). El uso de fdisk/sfdisk no figura en los apuntes disponibles.

4.3 /www_dir como raíz del sitio web

Se monta la partición de 3 GB, se copian los archivos del sitio y se asignan al usuario con el que corre Apache (`www-data`):

```
mount /dev/sdc1 /www_dir
cp /root/index.php /root/logo.png /www_dir/
chown -R www-data:www-data /www_dir
```

La consigna menciona dos archivos de Apache. En `/etc/apache2/sites-available/000-default.conf`, el sitio por defecto, se cambia la raíz de documentos:

```
<VirtualHost *:80>
        ServerAdmin webmaster@localhost
        DocumentRoot /www_dir

        ErrorLog ${APACHE_LOG_DIR}/error.log
        CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

Por seguridad, `/etc/apache2/apache2.conf` deniega el acceso a todo el sistema de archivos salvo los directorios autorizados explícitamente (como `/var/www`). Sin un permiso para `/www_dir`, Apache respondería *403 Forbidden*. Se agrega:

```
<Directory /www_dir/>
        Options Indexes FollowSymLinks
        AllowOverride None
        Require all granted
</Directory>
```

```
apache2ctl configtest          # debe responder "Syntax OK"
systemctl restart apache2
```

Con PHP instalado, `index.php` está incluido en la directiva `DirectoryIndex`, de modo que al pedir `http://192.168.0.200/` Apache lo ejecuta automáticamente.

> Fuente: consigna del TP (punto 4.3). La configuración de Apache corresponde a la documentación oficial de Apache y Debian.

4.4 y 4.5 Montaje automático de /www_dir y /backup_dir

Un `mount` manual se pierde al reiniciar. Para que las particiones se monten en cada arranque se declaran en `/etc/fstab`. Se las identifica por **UUID**, un identificador único del sistema de archivos que no cambia aunque cambie el orden de detección de los discos (lo que sí puede ocurrir con nombres como `/dev/sdc1`):

```
blkid /dev/sdc1 /dev/sdc2      # muestra el UUID de cada partición
```

```
# <dispositivo>                              <punto de montaje> <tipo> <opciones>
<dump> <pass>
UUID=5d774eb8-b548-433e-a588-45907763e0ad  /www_dir     ext4  defaults  0  2
UUID=1d9123b3-011d-40a4-a6ea-d21a53143d2a  /backup_dir  ext4  defaults  0  2
```

- **dump** (0): no se incluye en copias con la utilidad dump.
- **pass** (2): orden de verificación con `fsck` al arrancar; 1 se reserva para la raíz y 2 para las demás particiones.

```
mount -a                      # monta todo lo declarado en fstab (detecta errores
sin reiniciar)
df -h /www_dir /backup_dir
```

Antes de reiniciar conviene ejecutar `mount -a`: un error en `fstab` puede impedir el arranque normal del sistema. Tras reiniciar la VM, ambas particiones quedaron montadas automáticamente.

> Fuente: consigna del TP (puntos 4.4 y 4.5). La sintaxis de /etc/fstab corresponde a la documentación oficial de Debian.

5. Backup

5.1 Script backup_full.sh

El script se guarda en `/opt/scripts/backup_full.sh`, con permiso de ejecución (`chmod 755`). Cumple los requisitos de la consigna:

Requisito	Implementación
Recibe origen y destino como argumentos	Variables posicionales `$1` y `$2`; se exige exactamente dos argumentos (`$#`).
Opción de ayuda `-help`	Función `mostrar_ayuda`, que se ejecuta con `-help` (también acepta `--help` y `-h`).
Nombre con fecha ANSI	`date +%Y%m%d` genera AAAAMMDD; `basename` toma el último componente de la ruta: `/var/log` → `log_bkp_20261109.tar.gz`.
Validación del origen	Que exista, sea un directorio legible (`-d`, `-r`) y que su sistema de archivos esté montado (`findmnt -T`).
Validación del destino	Que exista, tenga permiso de escritura (`-w`) y sea un punto de montaje (`mountpoint -q`): así el backup nunca se escribe en el disco del sistema si `/backup_dir` no está montada.
Destino en la partición de /backup_dir	Las tareas programadas usan `/backup_dir` como destino.

```
#!/bin/bash
# backup_full.sh - Genera un backup comprimido (.tar.gz) de un directorio ORIGEN
# y lo guarda en un directorio DESTINO.
# Nombre del archivo: <nombre_origen>_bkp_<YYYYMMDD>.tar.gz

mostrar_ayuda() {
    echo "Uso: $0 <origen> <destino>"
    echo ""
    echo "Genera un backup comprimido del directorio <origen> y lo guarda en
<destino>."
    echo "El archivo se llama <nombre_origen>_bkp_<YYYYMMDD>.tar.gz"
    echo ""
    echo "Argumentos:"
    echo "  origen    Directorio que se quiere backupear (ej: /var/log)"
    echo "  destino   Directorio donde se guarda el backup (ej: /backup_dir)"
    echo ""
    echo "Opciones:"
    echo "  -help     Muestra esta ayuda"
}

# Opcion de ayuda
if [ "$1" = "-help" ] || [ "$1" = "--help" ] || [ "$1" = "-h" ]; then
    mostrar_ayuda
    exit 0
fi

# Validar cantidad de argumentos
if [ $# -ne 2 ]; then
    echo "ERROR: se necesitan 2 argumentos (origen y destino)."
    mostrar_ayuda
    exit 1
fi

ORIGEN="${1%/}"      # quita una barra final, si la hubiera
DESTINO="${2%/}"

# Origen: debe existir, ser directorio y poder leerse
if [ ! -d "$ORIGEN" ] || [ ! -r "$ORIGEN" ]; then
    echo "ERROR: el origen '$ORIGEN' no existe o no se puede leer."
    exit 2
fi

# Origen: su sistema de archivos debe estar montado
if ! findmnt -T "$ORIGEN" > /dev/null 2>&1; then
    echo "ERROR: el sistema de archivos del origen '$ORIGEN' no esta disponible."
    exit 2
fi

# Destino: debe existir y poder escribirse
if [ ! -d "$DESTINO" ] || [ ! -w "$DESTINO" ]; then
    echo "ERROR: el destino '$DESTINO' no existe o no se puede escribir."
    exit 3
```

```
fi

# Destino: debe ser un punto de montaje (la particion de backup)
if ! mountpoint -q "$DESTINO"; then
    echo "ERROR: el destino '$DESTINO' no es un sistema de archivos montado."
    exit 3
fi

FECHA=$(date +%Y%m%d)
NOMBRE=$(basename "$ORIGEN")
ARCHIVO="$DESTINO/${NOMBRE}_bkp_${FECHA}.tar.gz"

echo "Backup de '$ORIGEN' en '$ARCHIVO'..."
if tar -czf "$ARCHIVO" -C "$(dirname "$ORIGEN")" "$NOMBRE"; then
    echo "Backup finalizado correctamente."
    exit 0
else
    echo "ERROR: fallo la generacion del backup."
    exit 4
fi
```

Elementos del script

- `#!/bin/bash` (*shebang*): indica qué intérprete ejecuta el archivo.
- **Códigos de salida** (`exit 0` a `exit 4`): 0 indica éxito y cada valor distinto de 0 identifica un tipo de error, lo que permite a otro programa o a cron saber qué falló.
- `> /dev/null 2>&1` : descarta la salida estándar y la de errores de `findmnt` ; solo interesa su resultado (éxito o fallo). `/dev/null` es el dispositivo que descarta todo lo que recibe.
- `tar -czf` : `c` crea un archivo, `z` lo comprime con gzip y `f` indica el nombre del archivo. La opción `-C` cambia al directorio padre antes de empaquetar, de modo que el contenido queda guardado con ruta relativa (`log/…` en lugar de `/var/log/…`).

Pruebas realizadas

```
/opt/scripts/backup_full.sh -help                    # muestra la ayuda
/opt/scripts/backup_full.sh                          # ERROR: se necesitan 2
argumentos  (código 1)
/opt/scripts/backup_full.sh /noexiste /backup_dir    # ERROR: el origen no existe
(código 2)
/opt/scripts/backup_full.sh /www_dir /tmp/x          # ERROR: destino no montado
(código 3)
/opt/scripts/backup_full.sh /var/log /backup_dir     # genera
log_bkp_20261109.tar.gz
/opt/scripts/backup_full.sh /www_dir /backup_dir     # genera
www_dir_bkp_20261109.tar.gz
```

> La consigna escribe el formato como “YYYMMDD”, pero su propio ejemplo (`log_bkp_20240302`) tiene ocho dígitos: el formato ANSI correcto es AAAAMMDD. Del mismo modo, menciona “/var/logs”, mientras que en Debian el directorio de registros es `/var/log` , que es el que usa el ejemplo de la consigna.

> Fuente: consigna del TP (punto 5); apunte *Redireccionamiento y Pipes* (/dev/null, stderr, 2>&1); apunte *El standard FHS* (/opt, /var). No se dispone de apunte sobre programación de scripts en bash.

5.2 Programación con cron

`cron` es el servicio que ejecuta tareas periódicas. Cada usuario tiene su tabla de tareas (*crontab*), que se edita con `crontab -e` y se lista con `crontab -l`. Cada línea tiene cinco campos de tiempo seguidos del comando:

Minuto	Hora	Día del mes	Mes	Día de la semana	Comando
0–59	0–23	1–31	1–12	0–7 (0 y 7 = domingo)	ruta completa

Crontab de root:

```
0 0 * * * /opt/scripts/backup_full.sh /var/log /backup_dir
0 23 * * 1,3,5 /opt/scripts/backup_full.sh /www_dir /backup_dir
```

- Primera línea: minuto 0, hora 0, todos los días, todos los meses, cualquier día de la semana → **todos los días a las 00:00**.
- Segunda línea: minuto 0, hora 23, días de semana 1, 3 y 5 → **lunes, miércoles y viernes a las 23:00**.

Se usan rutas absolutas porque cron ejecuta las tareas con un entorno mínimo. Las tareas corren como root, que tiene permisos de lectura sobre `/var/log` y de escritura sobre `/backup_dir`. El servicio se verifica con `systemctl status cron`.

> Fuente: consigna del TP (punto 5.7). La sintaxis de crontab corresponde a la documentación oficial (`man 5 crontab`).

6. Entregables

Se creó un repositorio en GitHub con un `README.md` que lista a los integrantes del grupo. Cada directorio pedido se comprimió por separado en formato `.tar.gz`:

```
mkdir -p /home/entrega && cd /home/entrega
for d in root etc opt www_dir backup_dir; do
    tar -czpf "$d.tar.gz" -C / "$d"
done
```

La opción `p` preserva los permisos originales de los archivos. Los comprimidos se generan en `/home`, una partición distinta de los directorios que se empaquetan, para no incluir el archivo que se está creando dentro de sí mismo.

GitHub rechaza archivos de más de 100 MB, por lo que `/var` se comprime y se divide en partes de 20 MB con `split`, conectando ambos comandos con un pipe:

```
tar -czpf - -C / var | split -b 20M -d - var.tar.gz.part_
# genera var.tar.gz.part_00, var.tar.gz.part_01, ...
```

El guion `-` indica a `tar` que escriba en la salida estándar y a `split` que lea de la entrada estándar; `-b 20M` fija el tamaño de cada parte y `-d` numera los sufijos con dígitos. Para reconstruir el archivo original se concatenan las partes en orden:

```
cat var.tar.gz.part_* > var.tar.gz
tar -tzf var.tar.gz > /dev/null && echo "archivo íntegro"
```

Archivo	Contenido
`root.tar.gz`	/root: index.php, logo.png, db.sql, par de claves SSH y material adicional
`etc.tar.gz`	/etc: configuración de red, SSH, Apache, fstab, hostname y APT
`opt.tar.gz`	/opt: scripts/backup_full.sh y el archivo particion
`www_dir.tar.gz`	/www_dir: el sitio web
`backup_dir.tar.gz`	/backup_dir: los backups generados por el script
`var.tar.gz.part_00` a `part_03`	/var dividido en partes

> Fuente: consigna del TP (punto 6); apunte *Redireccionamiento y Pipes* (pipes y redirección de salida).

7. Verificación integral del trabajo

Punto	Comando de verificación	Resultado esperado
Hostname	`hostname`	TPServer
Versión	`cat /etc/debian_version`	12.x
SSH	Desde la PC: `ssh -i clave_privada.txt root@192.168.0.200`	Ingresa sin pedir contraseña
Web y base de datos	Navegador de la PC: `http://192.168.0.200`	Tabla de alumnos y notas con el logo
PHP	`php -v`	8.2.x (≥ 7.3)
Servicios	`systemctl is-active ssh apache2 mariadb cron`	active (×4)
IP estática	`ip -br a` y `cat /etc/network/interfaces`	192.168.0.200/24; campos address, netmask, gateway
Particiones	`fdisk -l /dev/sdc`	sdc1 3G y sdc2 6G, Id 83
Montaje automático	`reboot`, luego `df -h`	/www_dir y /backup_dir montados
Copia de /proc/partitions	`cat /opt/particion`	Tabla de particiones
Script	`/opt/scripts/backup_full.sh -help`	Texto de ayuda
Cron	`crontab -l`	Las dos tareas programadas

8. Preguntas de repaso

1. **¿Por qué `init=/bin/bash` permite cambiar la contraseña de root sin conocerla?** Porque el kernel ejecuta una shell de root en lugar del sistema de inicio, y por lo tanto nunca se llega a la pantalla de login que la pediría.
2. **¿Qué diferencia hay entre `apt upgrade` y `apt full-upgrade`?** `upgrade` no instala ni elimina paquetes; `full-upgrade` sí, y por eso es el que permite cambiar de versión mayor.
3. **¿Qué archivo determina de qué versión de Debian se descargan los paquetes?** `/etc/apt/sources.list`, a través del nombre de la distribución en cada línea.
4. **¿Dónde se guarda la clave pública que autoriza un ingreso por SSH?** En `~/.ssh/authorized_keys` del usuario con el que se ingresa en el servidor; la privada queda en el cliente.
5. **¿Qué cambia entre `PermitRootLogin yes` y `prohibit-password`?** Con `yes` root puede ingresar con contraseña o con clave; con `prohibit-password`, solo con clave.

6. **¿Por qué la VM usa adaptador puente?** Para estar en la misma red que la máquina física y recibir una IP de ese rango, lo que permite probar los servicios desde la PC.
7. **¿Por qué en `fstab` se usa UUID y no `/dev/sdc1`?** Porque el nombre del dispositivo depende del orden de detección y puede cambiar; el UUID identifica al sistema de archivos de forma permanente.
8. **¿Qué ocurriría si `/backup_dir` no estuviera montada y el script no lo validara?** El backup se escribiría en el directorio vacío del disco del sistema, ocupando espacio de la raíz y sin quedar en la partición destinada a backups.
9. **¿Por qué se guarda `/proc/partitions` en `/opt/particion`?** Porque `/proc` se genera en memoria y su contenido no persiste al apagar.
10. **¿Cómo se interpreta `0 23 * * 1,3,5`?** A las 23:00 de los lunes, miércoles y viernes.
11. **¿Por qué fue necesario agregar un bloque `<Directory>` en `apache2.conf`?** Porque Apache deniega por defecto el acceso fuera de los directorios autorizados; sin ese permiso, `/www_dir` respondería 403.
12. **¿Por qué se divide `/var` en partes?** Porque GitHub no admite archivos de más de 100 MB; con `split` se divide y con `cat` se reconstruye.

9. Bibliografía

- Universidad de Palermo. *Trabajo práctico integrador grupal – Consignas* (0306_TPIntegrador_261Q_V4).
- Universidad de Palermo. Apuntes de Computación Aplicada: *Descarga de Linux para la instalación*; *Comandos*; *El standard FHS*; *Redireccionamiento y Pipes*; *Gestor de paquetes*; *Redes*; *SSH – Secure Shell*; *Introducción a filesystem*.
- Niklas, P. (2020). *GNU/Linux, con sabor a Debian*. Primera edición.
- Debian. *Notas de publicación de Debian 12 (bookworm)*. https://www.debian.org/releases/bookworm/
- The Apache Software Foundation. *Apache HTTP Server 2.4 Documentation*. https://httpd.apache.org/docs/2.4/
- MariaDB Foundation. *MariaDB Documentation*. https://mariadb.com/kb/
- Páginas de manual de Debian: `fstab(5)`, `crontab(5)`, `fdisk(8)`, `sfdisk(8)`, `tar(1)`, `split(1)`, `sshd_config(5)`.
- Oracle. *VirtualBox*. https://www.virtualbox.org/